

241-405 Optimization

Transportation Problem 4

Traveling Salesman Problem (TSP)

Yossawee Keaomanee

# Outline

- Introduction to Traveling Salesman Problem
- Problem Formulation
- Exact Methods for Solving TSP
- Heuristic and Approximation Algorithms
- Case Studies and Applications
- Summary

# Introduction to Traveling Salesman Problem (TSP)

## What is TSP?

ปัญหาการของเดินทางพนักงานขาย (Traveling Salesman Problem: TSP) เป็นหนึ่งในโจทย์ปัญหาคลาสสิกที่สำคัญที่สุดในทางวิทยาการคอมพิวเตอร์ การวิจัยดำเนินการ (Operations Research) และการจัดการโลจิสติกส์

### แนวคิด:

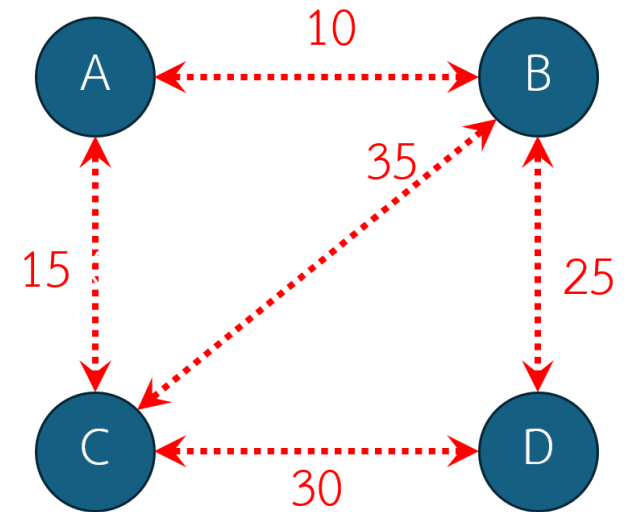
เมื่อกำหนดรายชื่อเมืองและระยะทางระหว่างแต่ละเมืองมาให้ ต้องการหาเส้นทางที่สั้นที่สุด ซึ่งสามารถเดินทางผ่านทุกเมืองได้ “เพียงครั้งเดียว” และสุดท้ายต้องกลับมายังเมืองเริ่มต้นอีกครั้ง

# Why is TSP important?

- ใช้จำลองสถานการณ์จริงได้หลากหลายรูปแบบ โดยเฉพาะงานที่เกี่ยวข้องกับการวางแผนเส้นทาง การจัดตารางงาน และระบบโลจิสติกส์
- ตัวอย่างการประยุกต์ใช้งาน
  - การลดต้นทุนโลจิสติกส์และคลังสินค้า (Real-world Impact)  
ใช้ค้นหาเส้นทางที่มีประสิทธิภาพสูงสุดสำหรับรถขนส่งสินค้า หรือพนักงานขายที่ต้องเดินทางหลายจุด
  - ด้านการผลิต (Manufacturing)  
ใช้เพิ่มประสิทธิภาพการเคลื่อนที่ของแขนกล หุ่นยนต์ หรือเครื่องมือตัดในสายการผลิต
  - ด้านเครือข่ายและอิเล็กทรอนิกส์ (Networking)  
ใช้ออกแบบการเดินสายไฟ หรือการจัดวางวงจรอิเล็กทรอนิกส์ให้มีประสิทธิภาพสูงสุด

# Simple Example

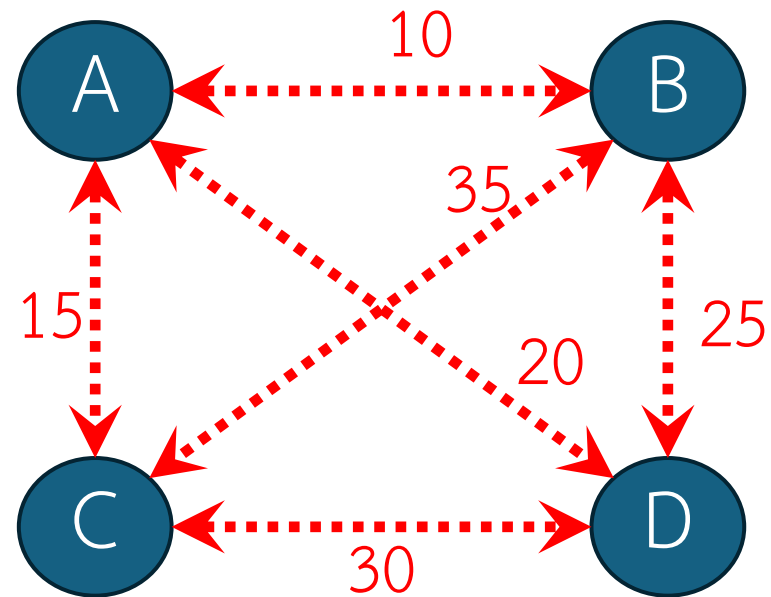
- มีเมือง 4 เมือง ได้แก่ A, B, C และ D
- ต้องการเริ่มต้นเดินทางจากเมือง A
  - จากนั้นเดินทางไปยังทุกเมืองที่เหลือให้ครบเพียงครั้งเดียว
  - สุดท้ายต้องเดินทางกลับมายังเมือง A อีกครั้ง
- เป้าหมายคือการหาเส้นทางที่มีระยะทางรวมสั้นที่สุด
  - ตัวอย่างเช่น:  $A \rightarrow C \rightarrow B \rightarrow D \rightarrow A$



# Problem Formulation

# Representing Cities as Graph Nodes

- แต่ละเมืองถูกแทนด้วยโหนด (Node หรือ Vertex) ภายในกราฟ
- เส้นทางระหว่างเมืองสองเมืองแทนด้วยเส้นเชื่อม (Edge) ที่เชื่อมต่อระหว่างโหนดทั้งสองเข้าด้วยกัน
- ระยะทางหรือต้นทุนระหว่างทุกคู่ของเมืองจะถูกจัดเก็บในรูปแบบเมทริกซ์ เรียกว่า **เมทริกซ์ระยะทาง (Distance Matrix)** หรือ **เมทริกซ์ต้นทุน (Cost Matrix)**.



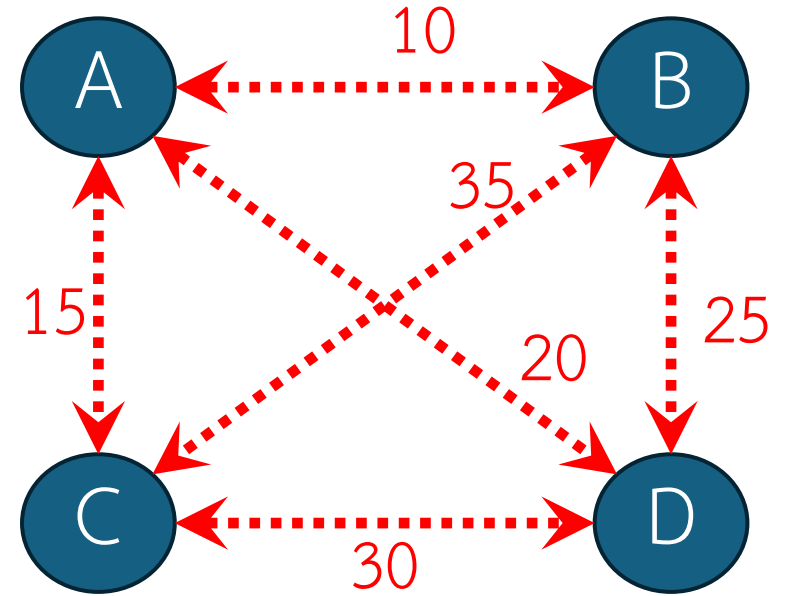
# Symmetric vs Asymmetric TSP

- ปัญหา TSP แบบสมมาตร (Symmetric TSP)
  - ระยะทางจากเมือง  $i$  ไปยังเมือง  $j$  มีค่าเท่ากับระยะทางจากเมือง  $j$  ไปยังเมือง  $i$  หรือเขียนได้เป็น  $c_{ij} = c_{ji}$
- ปัญหา TSP แบบอสมมาตร (Asymmetric TSP)
  - ระยะทางอาจแตกต่างกันตามทิศทางการเดินทาง เช่น ถนนเดินรถทางเดียว สภาพการจราจร หรือข้อจำกัดของเส้นทาง
  - ทำให้ระยะทางจากเมือง  $i$  ไปยังเมือง  $j$  ไม่จำเป็นต้องเท่ากับระยะทางจากเมือง  $j$  ไปยังเมือง  $i$



# Cost/Distance Matrix

- ตัวอย่าง มีเมืองทั้งหมด  $n$  เมือง เมทริกซ์ จะมีขนาด  $n \times n$  โดยสมาชิก  $c_{ij}$  แทน ค่าต้นทุนหรือระยะทางจากเมือง  $i$  ไปยังเมือง  $j$
- สำหรับสมาชิกบนแนวทแยงมุม  $c_{ii}$  มีค่าเป็นศูนย์ เนื่องจากระยะทางจากเมืองหนึ่งไปยังเมืองเดิมของตนเองมีค่าเท่ากับศูนย์



| From / To | A  | B  | C  | D  |
|-----------|----|----|----|----|
| A         | 0  | 10 | 15 | 20 |
| B         | 10 | 0  | 35 | 25 |
| C         | 15 | 35 | 0  | 30 |
| D         | 20 | 25 | 30 | 0  |

# Constraints and Assumptions

- เดินทางไปยังแต่ละเมืองให้ครบเพียงหนึ่งครั้งเท่านั้น (ห้ามเดินทางซ้ำ ยกเว้นการกลับมายังเมืองเริ่มต้น)
- จุดเริ่มต้นและจุดสิ้นสุดต้องเป็นเมืองเดียวกัน
- ทุกเมืองต้องสามารถเชื่อมต่อกันได้ (กล่าวคือ ต้องมีเส้นทางที่สามารถเดินทางระหว่างเมืองใด ๆ ได้)
- ค่าต้นทุนหรือระยะทางต้องเป็นค่าที่ไม่ติดลบ (Non-negative)

# Mathematical Formulation

กำหนดให้  $x_{ij}$  เป็นตัวแปรแบบไบนารี (Binary Variable):

$$x_{ij} = \begin{cases} 1 & \text{if the path goes directly from city } i \text{ to city } j \\ 0 & \text{otherwise} \end{cases}$$

ฟังก์ชันวัตถุประสงค์ (Objective):

$$\min \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

ภายใต้ข้อจำกัด:

- แต่ละเมืองต้องถูกเดินทางเข้า และ เดินทางออก เพียงหนึ่งครั้งเท่านั้น (Inflow - Outflow)
- ต้องไม่มีเส้นทางย่อย (Sub-tour)
- ต้องเป็นวงจรเดียว (Single Cycle) ที่ครอบคลุมทุกเมืองทั้งหมด

# Exact Methods for Solving TSP

**Exact Methods (วิธีแบบแม่นยำ)** อัลกอริทึมที่รับประกันได้ว่าจะหาคำตอบที่ดีที่สุด (Optimal Solution) ของปัญหา Traveling Salesman Problem (TSP).

- พิจารณาเส้นทางที่เป็นไปได้ทั้งหมด (All possible tours)
- เหมาะสำหรับปัญหาขนาดเล็ก โดยทั่วไปประมาณ **น้อยกว่า 20 เมือง**
- ใช้เวลาคำนวณเพิ่มขึ้นอย่างรวดเร็ว (Computational time grows rapidly) เมื่อจำนวนเมืองเพิ่มขึ้นเนื่องจากความซับซ้อนของปัญหาเพิ่มแบบทวีคูณ.

# Brute Force (Exhaustive Search)

- สร้างเส้นทางที่เป็นไปได้ทั้งหมด (Permutations ของเมืองทั้งหมด)
- คำนวณต้นทุนรวมของแต่ละเส้นทาง
- เลือกเส้นทางที่มีต้นทุนรวมต่ำที่สุด (Minimum total cost).

จำนวนเส้นทางที่เป็นไปได้:  $(n - 1)!$

- สำหรับปัญหา TSP แบบสมมาตร (Symmetric TSP) จะพิจารณาลดจำนวนเส้นทางลง เนื่องจาก
  - การเดินทางเป็นวงรอบ (Round trip)
  - เส้นทางที่ย้อนทิศทางกันถือว่าเหมือนกัน (Direction symmetry)

# Brute Force (Exhaustive Search)

## ข้อดี (Pros)

- สามารถหาคำตอบที่เหมาะสมที่สุด (Optimal Solution) ได้เสมอ.

## ข้อเสีย (Cons)

- ใช้ทรัพยากรในการคำนวณสูงมาก จนไม่สามารถนำไปใช้กับปัญหาขนาดใหญ่ได้ (Computationally infeasible) เมื่อจำนวน  $n$  มีค่ามาก.

## ตัวอย่าง:

- 10 เมือง  $\rightarrow 9!/2=181,440$
- 20 เมือง  $\rightarrow$  มีจำนวนเส้นทางมากถึงระดับ “ล้านล้านเส้นทาง”

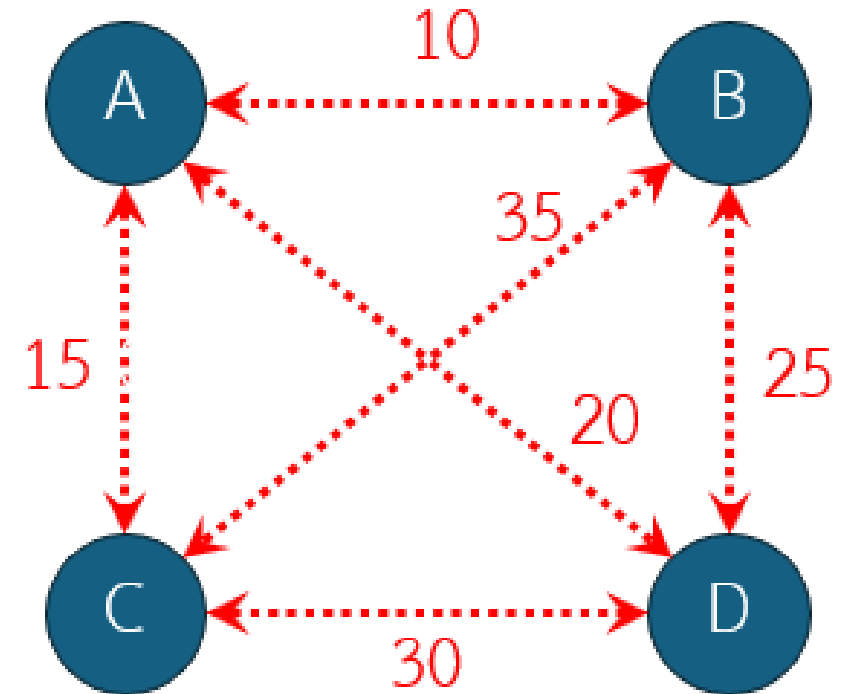
# Brute Force Example

## Problem Setup

- เมือง: A, B, C, D
- เริ่มต้นและจบที่เมือง A

## Distance Matrix:

| From\To | A  | B  | C  | D  |
|---------|----|----|----|----|
| A       | 0  | 10 | 15 | 20 |
| B       | 10 | 0  | 35 | 25 |
| C       | 15 | 35 | 0  | 30 |
| D       | 20 | 25 | 30 | 0  |



# Brute Force Example

ขั้นตอนที่ 1: แสดงรายการเส้นทางที่เป็นไปได้ทั้งหมด

- กรณีที่มี 4 เมือง จำนวนเส้นทางทั้งหมดคือ:  $(n - 1)! = (4 - 1)! = 6$

| Tour # | Route                                                       |
|--------|-------------------------------------------------------------|
| 1      | $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ |
| 2      | $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$ |
| 3      | $A \rightarrow C \rightarrow B \rightarrow D \rightarrow A$ |
| 4      | $A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$ |
| 5      | $A \rightarrow D \rightarrow B \rightarrow C \rightarrow A$ |
| 6      | $A \rightarrow D \rightarrow C \rightarrow B \rightarrow A$ |

เส้นทางที่ย้อนทิศทางกันถือว่าเป็น  
เหมือนกัน (Direction symmetry)  
จำนวนเส้นทางจะเหลือ  $(n - 1)!/2$



# Brute Force Example

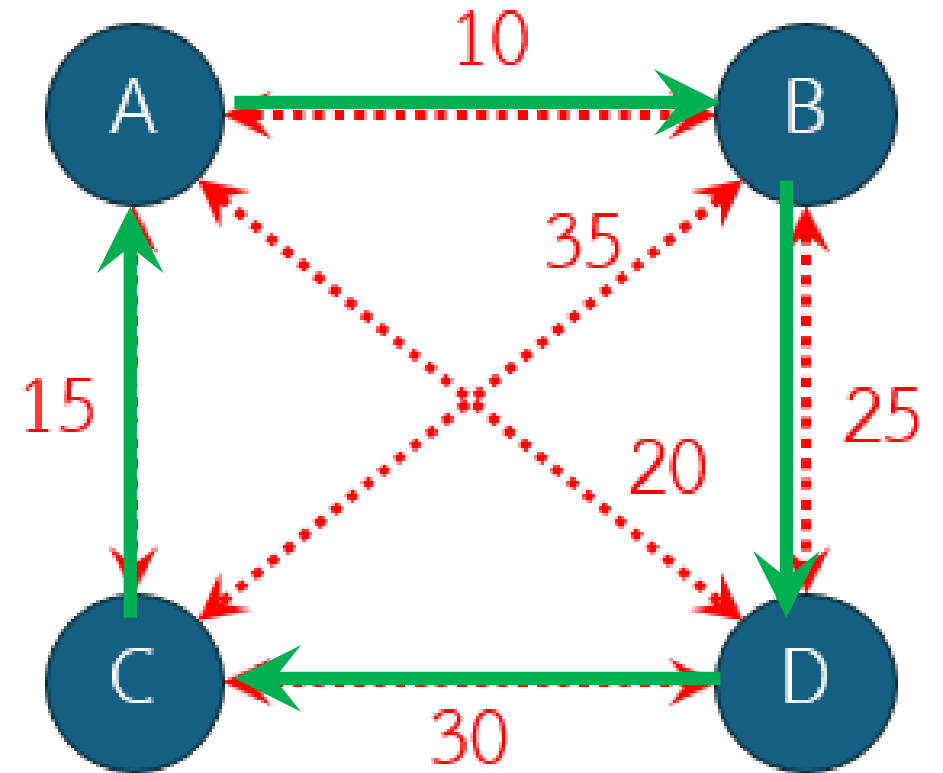
- ขั้นตอนที่ 2: คำนวณระยะทางรวมของแต่ละเส้นทาง

| Tour # | Route               | Total Distance           |
|--------|---------------------|--------------------------|
| 1      | $A - B - C - D - A$ | $10 + 35 + 30 + 20 = 95$ |
| 2      | $A - B - D - C - A$ | $10 + 25 + 30 + 15 = 80$ |
| 3      | $A - C - B - D - A$ | $15 + 35 + 25 + 20 = 95$ |
| 4      | $A - C - D - B - A$ | $15 + 30 + 25 + 10 = 80$ |
| 5      | $A - D - B - C - A$ | $20 + 25 + 35 + 15 = 95$ |
| 6      | $A - D - C - B - A$ | $20 + 30 + 35 + 10 = 95$ |

# Brute Force Example

ขั้นตอนที่ 3: หาทางเดินที่สั้นที่สุด (Find the Shortest Tour)

- ระยะทางรวมที่น้อยที่สุด (Min total distance) = 80
- เส้นทางที่เหมาะสมที่สุด (Optimal routes):
  - $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$
  - $A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$



# Dynamic Programming (Held-Karp Algorithm)

## แนวคิด (Idea):

- แบ่งปัญหา TSP (Travelling Salesman Problem) ออกเป็นปัญหาย่อยที่มีการซ้อนทับกัน (overlapping subproblems).

## แนวทาง (Approach):

- ใช้ความสัมพันธ์แบบเรียกซ้ำ (recursive relations) เพื่อคำนวณต้นทุนขั้นต่ำของการเดินทางบางส่วน (partial tours)
- จากนั้นนำผลลัพธ์ของปัญหาย่อยมาใช้สร้างคำตอบของเส้นทางที่ใหญ่ขึ้นทีละขั้น (build up solutions)

## ความซับซ้อนของเวลา (Time Complexity): $O(n^2 2^n)$

- มีประสิทธิภาพมากกว่าวิธี brute force :  $O(N!)$
- ยังคงมีความซับซ้อนแบบเอ็กซ์โพเนนเชียล (exponential)

# Dynamic Programming (Held-Karp Algorithm)

## ข้อดี (Pros):

- สามารถจัดการปัญหาขนาดใหญ่ได้ดีกว่าวิธีการแบบ brute force (การลองทุกความเป็นไปได้)

## ข้อจำกัด (Cons):

- ยังไม่เหมาะสำหรับปัญหาที่มีขนาดใหญ่มาก (เช่น  $n > 30$ ) เนื่องจากยังมีความซับซ้อนสูงและใช้ทรัพยากรมาก

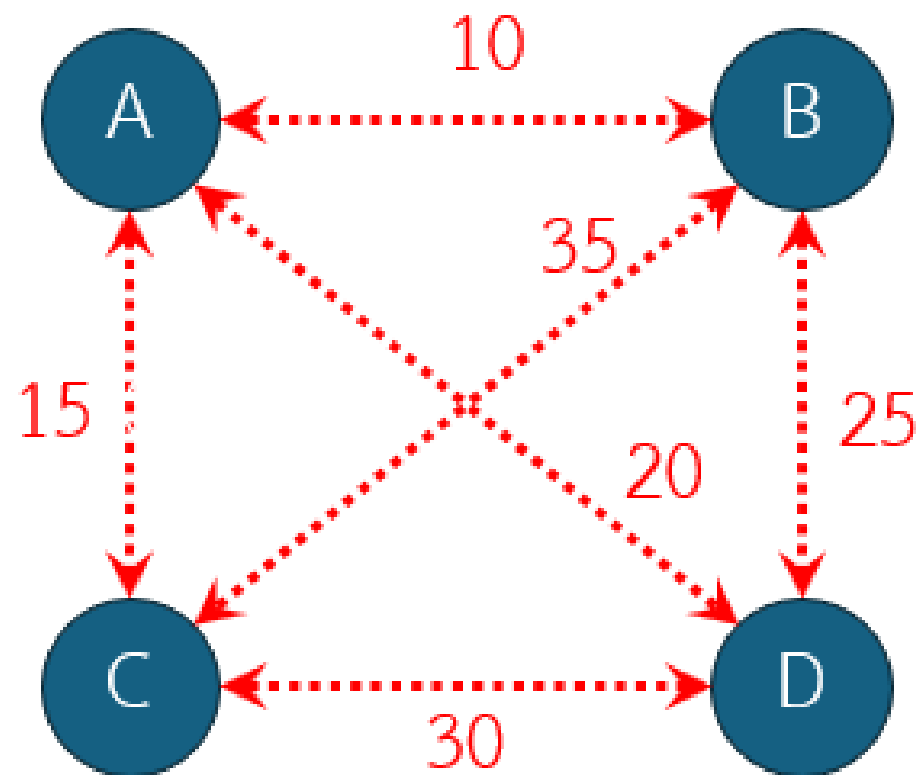
# Dynamic Programming Example

## การกำหนดปัญหา (Problem Setup)

- เมือง (Cities): A, B, C, D
- เริ่มต้นและสิ้นสุดการเดินทางที่เมือง A

## เมทริกซ์ระยะทาง (Distance Matrix):

| From\To | A  | B  | C  | D  |
|---------|----|----|----|----|
| A       | 0  | 10 | 15 | 20 |
| B       | 10 | 0  | 35 | 25 |
| C       | 15 | 35 | 0  | 30 |
| D       | 20 | 25 | 30 | 0  |



# Dynamic Programming Example

สมการหลัก (Main Equation)

$$Cost(S, j) = \min_{i \in S} \{Cost(S - \{i\}, i) + cost(i, j)\}$$

สัญลักษณ์:

- $Cost(S, j)$ : ระยะทางที่สั้นที่สุดในการเยี่ยมชมเมืองในเซต  $S$  ให้ครบ แล้วจบลงที่เมือง  $j$
- $cost(i, j)$ : ระยะทางตรงจากเมือง  $i$  ไปยังเมือง  $j$
- $S$ : เซตของเมืองย่อยที่ผ่านไปแล้ว (ไม่รวมจุดเริ่มต้น  $A$  และเมืองปลายทาง  $j$ )
- $j$ : เมืองปลายทางที่เรา กำลังจะเดินทางไปยังอยู่ ณ ปัจจุบัน
- $i$ : เมืองสุดท้ายก่อนหน้านี้ ที่อยู่ในเซต  $S$

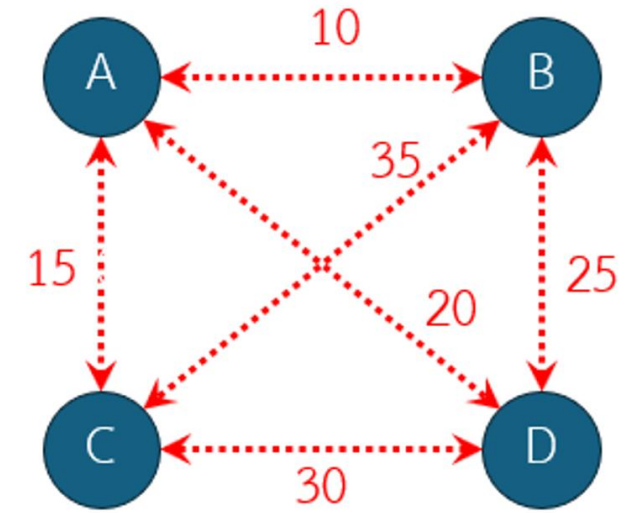
# Dynamic Programming Example

- ขั้นตอนที่ 1: เริ่มต้น (Initialization)
- คำนวณระยะทางที่สั้นที่สุดจากจุด A ไปยังจุดอื่น ๆ โดยตรง

$$Cost(\{\}, j) = cost(A, j)$$

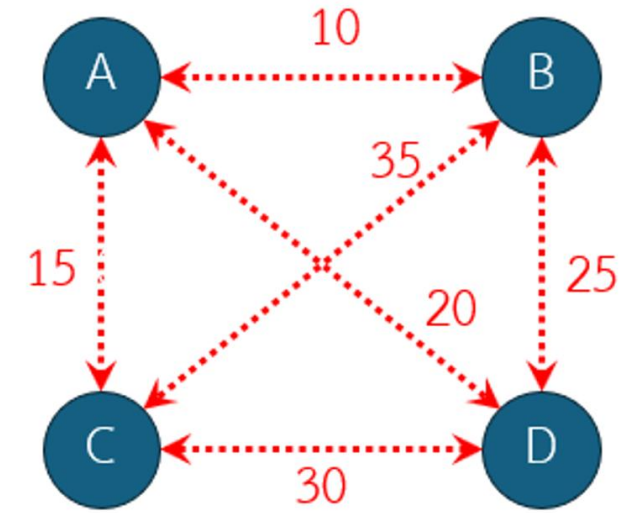
- ความหมาย:

- เมื่อยังไม่ได้ผ่านเมืองย่อยใดๆ เลย ( $S = \emptyset$  หรือเซตว่าง)
- ระยะทางเริ่มต้นไปยังเมือง  $j$  ใดๆ คือ ระยะทางตรงจากจุดเริ่มต้น  $A$  ไปยังเมือง  $j$



# Dynamic Programming Example

- ขั้นตอนที่ 1: เริ่มต้น (Initialization) ( $S = \emptyset$ )
- คำนวณระยะทางที่สั้นที่สุดจากจุด A ไปยังจุดอื่น ๆ โดยตรง
$$Cost(\{ \}, j) = cost(A, j)$$
- จากกราฟ จะได้:
  - $Cost(\emptyset, B) = cost(A, B) = \mathbf{10}$
  - $Cost(\emptyset, C) = cost(A, C) = \mathbf{15}$
  - $Cost(\emptyset, D) = cost(A, D) = \mathbf{20}$





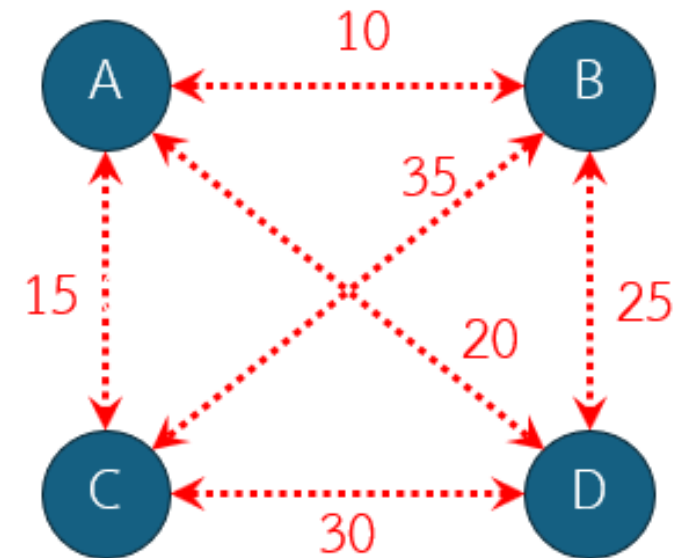
# Dynamic Programming Example

ขั้นตอนที่ 2: หาค่าระยะทางที่สั้นที่สุดสำหรับ Set ที่มี 1 เมือง (B C หรือ D)

- หาค่าเส้นทางจากเมือง A ผ่านเมืองหนึ่งเมืองในเซตย่อย  $S$  ไปยังเมือง  $j$  ใดๆ
- สมการ:  $Cost(\{i\}, j) = Cost(\{\}, i) + cost(i, j)$
- $i = B$

ผลลัพธ์: สำหรับ  $S = \{B\}$  และ  $j = C, D$

- $Cost(\{B\}, C) = Cost(\{\}, B) + cost(B, C)$   
 $= 10 + 35 = 45$
- $Cost(\{B\}, D) = Cost(\{\}, B) + cost(B, D)$   
 $= 10 + 25 = 35$



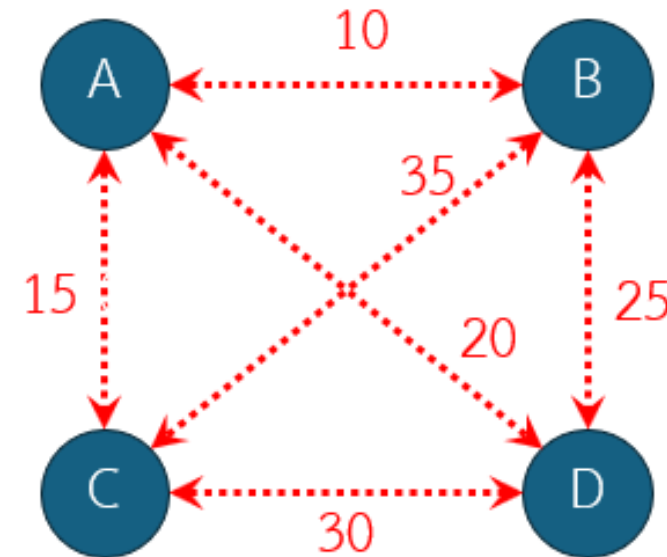
# Dynamic Programming Example

ขั้นตอนที่ 2: หาค่าระยะทางที่สั้นที่สุดสำหรับ Set ที่มี 1 เมือง (B C หรือ D)

- หาค่าเส้นทางจากเมือง A ผ่านเมืองหนึ่งเมืองในเซตย่อย  $S$  ไปยังเมือง  $j$
- สมการ:  $Cost(\{i\}, j) = Cost(\{\}, i) + cost(i, j)$
- $i = C$

ผลลัพธ์: สำหรับ  $S = \{C\}$  : และ  $j = B, D$

- $Cost(\{C\}, B) = Cost(\{\}, C) + cost(C, B)$   
 $= 15 + 35 = 50$
- $Cost(\{C\}, D) = Cost(\{\}, C) + cost(C, D)$   
 $= 15 + 30 = 45$



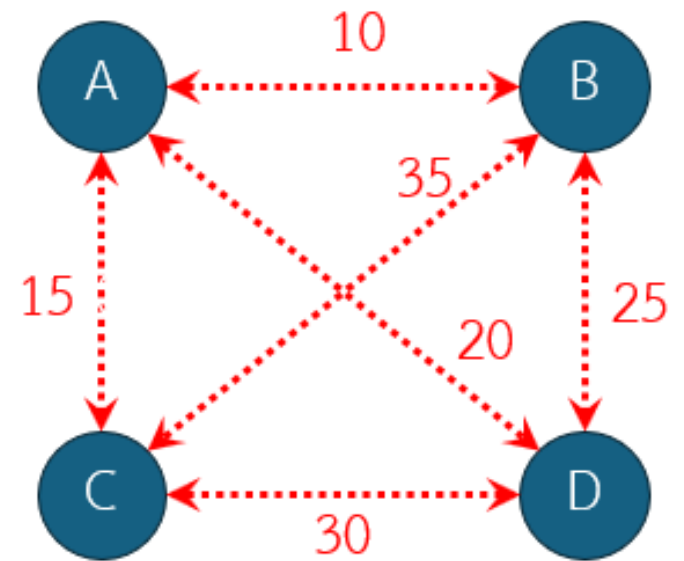
# Dynamic Programming Example

ขั้นตอนที่ 2: หาค่าระยะทางที่สั้นที่สุดสำหรับ Set ที่มี 1 เมือง (B C หรือ D)

- หาค่าเส้นทางจากเมือง A ผ่านเมืองหนึ่งเมืองในเซตย่อย  $S$  ไปยังเมือง  $j$
- สมการ:  $Cost(\{i\}, j) = Cost(\{ \}, i) + cost(i, j)$
- $i = D$

ผลลัพธ์: สำหรับ  $S = \{D\}$  : และ  $j = B, C$

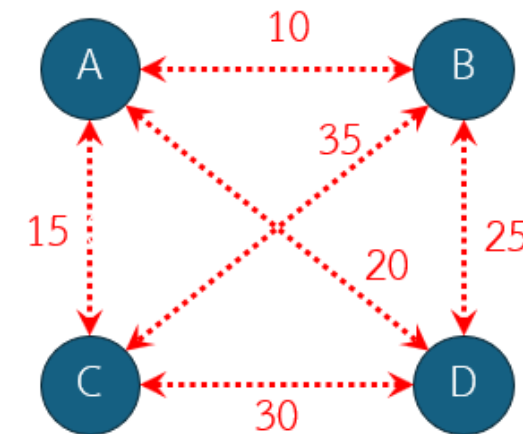
- $Cost(\{D\}, B) = Cost(\{ \}, D) + cost(D, B)$   
 $= 20 + 25 = 45$
- $Cost(\{D\}, C) = Cost(\{ \}, D) + cost(D, C)$   
 $= 20 + 30 = 50$



# Dynamic Programming Example

| เซต $S$ และปลายทาง $j$ | ค่า Cost | เมืองก่อนหน้า ( $i$ ) |
|------------------------|----------|-----------------------|
| $Cost(\{B\}, C)$       | 45       | $B$                   |
| $Cost(\{B\}, D)$       | 35       | $B$                   |
| $Cost(\{C\}, B)$       | 50       | $C$                   |
| $Cost(\{C\}, D)$       | 45       | $C$                   |
| $Cost(\{D\}, B)$       | 45       | $D$                   |
| $Cost(\{D\}, C)$       | 50       | $D$                   |

# Dynamic Programming Example



ขั้นตอนที่ 3: ค้นหาเส้นทางสำหรับชุดเมืองจำนวน 2 เมือง

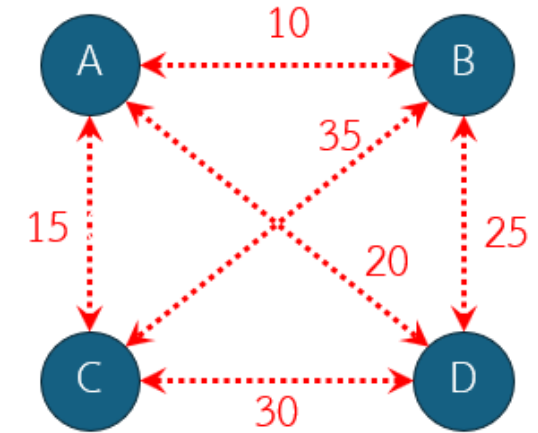
- ค้นหาเส้นทางจากเมือง  $A$  ผ่านชุดเมือง  $S$  ที่มี 2 เมือง และไปสิ้นสุดที่เมือง  $j$  โดยที่  $j$  ต้องเป็นเมืองที่ **ไม่อยู่ในชุด  $S$**
- สมการ  $Cost(S, j) = \min_{i \in S} \{Cost(S - \{i\}, i) + cost(i, j)\}$

ถ้า  $S = \{B, C\}$  (สิ้นสุดที่เมือง  $D$ )

ผลลัพธ์:

- เลือก  $i = B$ :  $Cost(\{C\}, B) + cost(B, D) = 50 + 25 = 75$
- เลือก  $i = C$ :  $Cost(\{B\}, C) + cost(C, D) = 45 + 30 = 75$
- สรุป:  $Cost(\{B, C\}, D) = \min(75, 75) = 75$

# Dynamic Programming Example



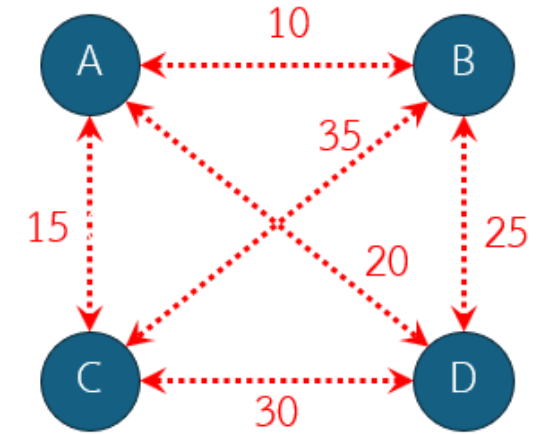
ขั้นตอนที่ 3: ค้นหาเส้นทางสำหรับชุดเมืองจำนวน 2 เมือง

- ค้นหาเส้นทางจากเมือง A ผ่านชุดเมือง  $S$  ที่มี 2 เมือง และไปสิ้นสุดที่เมือง  $j$  โดยที่  $j$  ต้องเป็นเมืองที่ **ไม่อยู่ในชุด  $S$**
- สมการ  $Cost(S, j) = \min_{i \in S} \{Cost(S - \{i\}, i) + cost(i, j)\}$

ผลลัพธ์:  $S = \{B, D\}$  (สิ้นสุดที่เมือง C)

- เลือก  $i = B$ :  $Cost(\{D\}, B) + cost(B, C) = 45 + 35 = 80$
- เลือก  $i = D$ :  $Cost(\{B\}, D) + cost(D, C) = 35 + 30 = 65$
- สรุป :  $Cost(\{B, D\}, C) = \min(80, 65) = 65$

# Dynamic Programming Example



ขั้นตอนที่ 3: ค้นหาเส้นทางสำหรับชุดเมืองจำนวน 2 เมือง

- ค้นหาเส้นทางจากเมือง  $A$  ผ่านชุดเมือง  $S$  ที่มี 2 เมือง และไปสิ้นสุดที่เมือง  $j$  โดยที่  $j$  ต้องเป็นเมืองที่ **ไม่อยู่ในชุด  $S$**
- สมการ  $Cost(S, j) = \min_{i \in S} \{Cost(S - \{i\}, i) + cost(i, j)\}$

ผลลัพธ์:  $S = \{C, D\}$  (สิ้นสุดที่เมือง  $B$ )

- เลือก  $i = C$ :  $Cost(\{D\}, C) + cost(C, B) = 50 + 35 = 85$
- เลือก  $i = D$ :  $Cost(\{C\}, D) + cost(D, B) = 45 + 25 = 70$
- สรุป:  $Cost(\{C, D\}, B) = \min(85, 70) = 70$

# Dynamic Programming Example

ขั้นตอนที่ 4: หาระยะทางกลับไปยังเมือง A (การปิดทัวร์ให้สมบูรณ์)

- คำนวณหาระยะทางที่สั้นที่สุดในการเดินทางผ่านทุกเมือง และกลับมายังเมือง A
- จากผลลัพธ์ในขั้นตอนที่ 3:
  - เมืองสุดท้ายคือ D:  $C(\{B, C\}, D) + cost(D, A) = 75 + 20 = 95$
  - เมืองสุดท้ายคือ C:  $C(\{B, D\}, C) + cost(C, A) = 65 + 15 = 80$
  - เมืองสุดท้ายคือ B:  $C(\{C, D\}, B) + cost(B, A) = 70 + 10 = 80$



# Dynamic Programming Example

สรุป: ระยะทางที่สั้นที่สุดจากจุด A ผ่านทุกเมืองและกลับมายังจุด A คือ 80 หน่วย

- $A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$
- $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$

# Limitations of Exact Methods

| วิธีการ                                   | ข้อดี                                               | ข้อเสีย                                        |
|-------------------------------------------|-----------------------------------------------------|------------------------------------------------|
| Brute Force (วิธีลอง<br>ทุกความเป็นไปได้) | ให้คำตอบที่เหมาะสมที่สุด<br>(Optimal solution) จริง | ใช้เวลามากมาก (ความซับซ้อนแบบ<br>แฟกทอเรียล)   |
| Dynamic<br>Programming                    | เร็วกว่าวิธี Brute Force                            | ยังมีความซับซ้อนของเวลาแบบเอ็กซ์<br>โพเนนเชียล |

# Exact/Direct Algorithms สำหรับ TSP

| อัลกอริทึม (Algorithm)           | กลไกการทำงานหลัก (Core Mechanism)                                                      | จุดเด่น (Strengths)                                                         | ข้อจำกัด (Limitations)                                                |
|----------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Branch and Bound                 | สร้าง Search Tree หาเส้นทางพร้อมคำนวณ Lower Bound เพื่อตัดกิ่งที่ไม่ใช่                | ลดพื้นที่การค้นหาได้ดีกว่า Brute Force และประหยัด RAM กว่า Held-Karp        | ถ้าประเมิน Bound ไม่ดี เวลาคำนวณจะแย่งไปใกล้เคียง Brute Force         |
| Integer Linear Programming (ILP) | แปลงโจทย์เป็นสมการเส้นตรงและข้อจำกัด (รวม Sub-tour Elimination) แล้วใช้ Solver หาคำตอบ | กำหนดข้อจำกัด เพิ่มเติมได้ง่าย ใช้ไลบรารีมาตรฐาน (Gurobi , CPLEX) แก้ได้เลย | การเพิ่มข้อจำกัดขจัด Sub-tour ทำให้จำนวนสมการเพิ่มขึ้นแบบ Exponential |
| Branch and Cut                   | ผสม Branch and Bound เข้ากับ Cutting Planes (บีบขอบเขตด้วยสมการเส้นตรงระหว่างหาคำตอบ)  | มีประสิทธิภาพสูงสุดในกลุ่ม Exact Method                                     | มีความซับซ้อนสูงมากในการ Implement                                    |

# Heuristic and Approximation Algorithms

# Why Heuristics?

- วิธีที่แน่นอน (Exact methods) จะไม่สามารถนำไปใช้ได้จริงกับปัญหา TSP ขนาดใหญ่ เนื่องจากมีความซับซ้อนของเวลาแบบแฟกทอเรียล/เอ็กซ์โพเนนเชียล
- วิธีฮิวริสติก (Heuristics) สามารถให้คำตอบที่ดีได้อย่างรวดเร็ว แต่ไม่รับประกันว่าจะเป็นคำตอบที่เหมาะสมที่สุด
- อัลกอริทึมเชิงประมาณค่า (Approximation algorithms) ให้คำตอบที่อยู่ภายในขอบเขตความคลาดเคลื่อนที่ทราบเมื่อเทียบกับคำตอบที่ดีที่สุด

# Common Heuristic Methods : Nearest Neighbor (NN)

## แนวคิด (Idea):

- เริ่มต้นจากเมืองใดเมืองหนึ่งแบบสุ่ม จากนั้นในแต่ละขั้นตอนให้เดินทางไปยังเมืองที่ยังไม่เคยถูกเยี่ยมชมและอยู่ใกล้ที่สุด
- ทำซ้ำกระบวนการนี้จนกว่าจะเยี่ยมชมครบทุกเมือง แล้วจึงกลับไปยังเมืองเริ่มต้น

## ข้อดี (Pros):

- ทำงานได้รวดเร็วและมีความซับซ้อนต่ำ ( $O(n^2)$ ).
- เข้าใจและนำไปใช้งานได้ง่าย

## ข้อเสีย (Cons):

- อาจให้ผลลัพธ์ที่ไม่ดีนักในบางรูปแบบของการจัดวางเมือง (เส้นทางไม่เหมาะสมหรือไม่สั้นที่สุด)

# Common Heuristic Methods Cheapest Insertion (วิธีการแทรกแบบต้นทุนต่ำที่สุด)

## แนวคิด (Idea):

- เริ่มต้นด้วยเส้นทางขนาดเล็ก (เช่น มี 2 เมือง) จากนั้นค่อย ๆ เพิ่มเมืองที่เหลือทีละเมือง โดยเลือกตำแหน่งในการแทรกเมืองใหม่ลงในเส้นทางปัจจุบันให้เกิดการเพิ่มของต้นทุน (ระยะทาง) น้อยที่สุดในแต่ละครั้ง

## ข้อดี (Pros):

- ในหลายกรณีให้ผลลัพธ์ที่ดีกว่า Nearest Neighbor (NN)

## ข้อเสีย (Cons):

- ใช้เวลาคำนวณมากกว่า Nearest Neighbor ( $O(n^2)$ ).

# Common Heuristic Methods : Genetic Algorithm (GA)

## แนวคิด (Idea):

- ได้รับแรงบันดาลใจจากระบบการคัดเลือกตามธรรมชาติ (natural selection) โดยเริ่มจากกลุ่มคำตอบ (เส้นทางการเดินทาง) แบบสุ่ม จากนั้นทำการคัดเลือก (selection), ผสมพันธุ์ (crossover) และกลายพันธุ์ (mutation) เพื่อพัฒนาให้ได้คำตอบที่ดีขึ้นเรื่อย ๆ

## ข้อดี (Pros):

- เหมาะสำหรับปัญหา TSP ขนาดใหญ่และซับซ้อน

## ข้อเสีย (Cons):

- ต้องปรับค่าพารามิเตอร์อย่างระมัดระวังเพื่อให้ได้ผลลัพธ์ที่ดี



# Common Heuristic Methods : Simulated Annealing (SA)

## การอบอ่อนจำลอง

### แนวคิด (Idea):

- เริ่มต้นด้วยคำตอบเริ่มต้น (Initial solution) หนึ่งค่า
- ในแต่ละรอบของการทำซ้ำ (iteration) จะมีการปรับเปลี่ยนเส้นทางเล็กน้อยแบบสุ่ม
- ยอมรับคำตอบที่แย่กว่าเดิมได้ โดยมีความน่าจะเป็นลดลงตามเวลา (เพื่อช่วยหลีกเลี่ยงการติดอยู่ที่คำตอบเฉพาะจุด หรือ local minima)

### ข้อดี (Pros):

- สามารถหลีกเลี่ยงการติดอยู่ที่คำตอบเฉพาะจุด (local optima) ได้ดี

### ข้อเสีย (Cons):

- อาจยังใช้เวลานานเมื่อปัญหา TSP มีขนาดใหญ่มาก

# Common Heuristic Methods : Ant Colony Optimization (ACO)

## แนวคิด (Idea):

- ได้รับแรงบันดาลใจจากพฤติกรรมของมดที่สามารถค้นหาเส้นทางที่สั้นที่สุดโดยใช้ “ฟีโรโมน” (pheromone trails) ในการสื่อสารกัน
- ในเชิงคอมพิวเตอร์ จะใช้ “มดจำลอง” (artificial ants) เพื่อสำรวจเส้นทางต่าง ๆ และจะเพิ่มความเข้มข้นของฟีโรโมนเสมือน ในเส้นทางที่ดี เพื่อส่งเสริมให้เส้นทางนั้นมีโอกาสถูกเลือกมากขึ้นในรอบถัดไป

## ข้อดี (Pros):

- สามารถประมวลผลแบบขนานได้ดี (Highly parallelizable)
- มีความยืดหยุ่นและปรับตัวได้ดี (Adaptive)

## ข้อเสีย (Cons):

- ใช้ทรัพยากรในการคำนวณสูง (Computationally intensive)

# Common Heuristic Methods

| อัลกอริทึม                                                 | ความเร็ว | คุณภาพของคำตอบ | ความสามารถในการขยายขนาด |
|------------------------------------------------------------|----------|----------------|-------------------------|
| Nearest Neighbor (วิธีเพื่อนบ้านใกล้ที่สุด)                | เร็ว     | ปานกลาง        | สูง                     |
| Genetic Algorithm (อัลกอริทึมเชิงพันธุกรรม)                | ปานกลาง  | ดี             | สูง                     |
| Simulated Annealing (การจำลองการอบอ่อน)                    | ปานกลาง  | ดี             | สูง                     |
| Ant Colony Optimization (การเพิ่มประสิทธิภาพแบบอาณานิคมมด) | ช้า      | ดีเยี่ยม       | ปานกลาง                 |

# Case Studies and Applications

## Logistics and Transportation

- ปรับเส้นทางการจัดส่งให้มีประสิทธิภาพสูงสุดสำหรับรถบรรทุก โดรน หรือพนักงานขาย
- ตัวอย่าง: รถบรรทุกขนส่งที่ต้องแวะส่งสินค้าในมากกว่า 50 จุดต่อวัน โดยมีเป้าหมายเพื่อลดระยะทางการเดินทาง ลดการใช้น้ำมัน และประหยัดเวลาในการขนส่ง

# Case Studies and Applications

## Manufacturing and Industrial Processes

- เพิ่มประสิทธิภาพเส้นทางการเคลื่อนที่ของเครื่องมือในกระบวนการกัด/กลึงด้วย CNC เพื่อให้ใช้เวลาการเคลื่อนที่ให้น้อยที่สุด
- ตัวอย่าง:  
การเจาะรูบนแผงวงจร (Circuit Board) ให้ใช้เวลารวมน้อยที่สุด โดยจัดลำดับตำแหน่งการเจาะให้เหมาะสมที่สุด

# Case Studies and Applications

## Robotics Path Planning

- วางแผนการเคลื่อนที่ของแขนกลหุ่นยนต์หรือยานพาหนะอัตโนมัติให้มีประสิทธิภาพสูงสุด
- ตัวอย่าง: หุ่นยนต์ในคลังสินค้าที่ทำหน้าที่หยิบสินค้า (Picking) จากชั้นวางหลายตำแหน่งอย่างเป็นระบบและรวดเร็ว